

# Elementos de Sistemas

## Unidade Lógica Aritmética

*"Contar é a religião desta geração, sua esperança e salvação."*

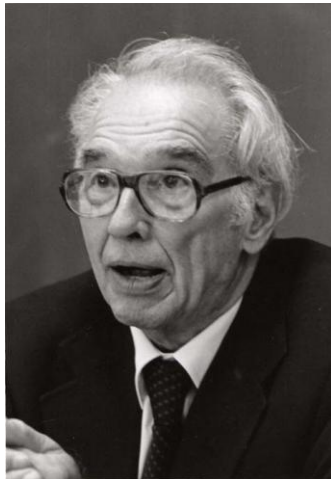
*"Counting is the religion of this generation, its hope and salvation."*

*Gertrude Stein (1874–1946)* escritora americana

apud Nisan, N. & Schocken, S. 2005. Elements of Computing Systems

# John Vincent Atanasoff & Clifford Berry

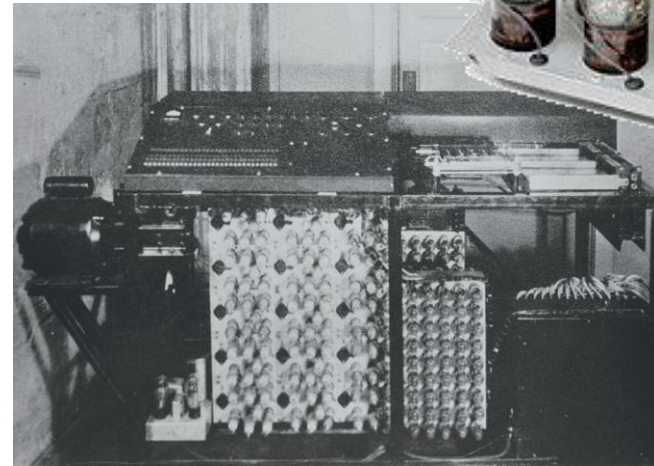
John Atanasoff e Clifford Berry implementaram na Iowa State University em 1939 o primeiro somador de 16-bits a válvula, e em 1941 fizeram uma máquina para resolver equações lineares, chamado de ABC (Atanasoff-Berry Computer), com 60 posições de memória de 50-bit feitas de capacitores, rodando a 60Hz. O julgamento Honeywell v. Sperry Rand declarou inválida a patente do ENIAC e reconheceu que Mauchly havia obtido ideias a partir do trabalho de Atanasoff.



John Atanasoff

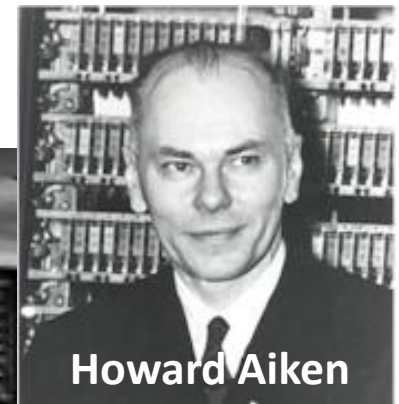


Clifford Berry



# Harvard Mark 1

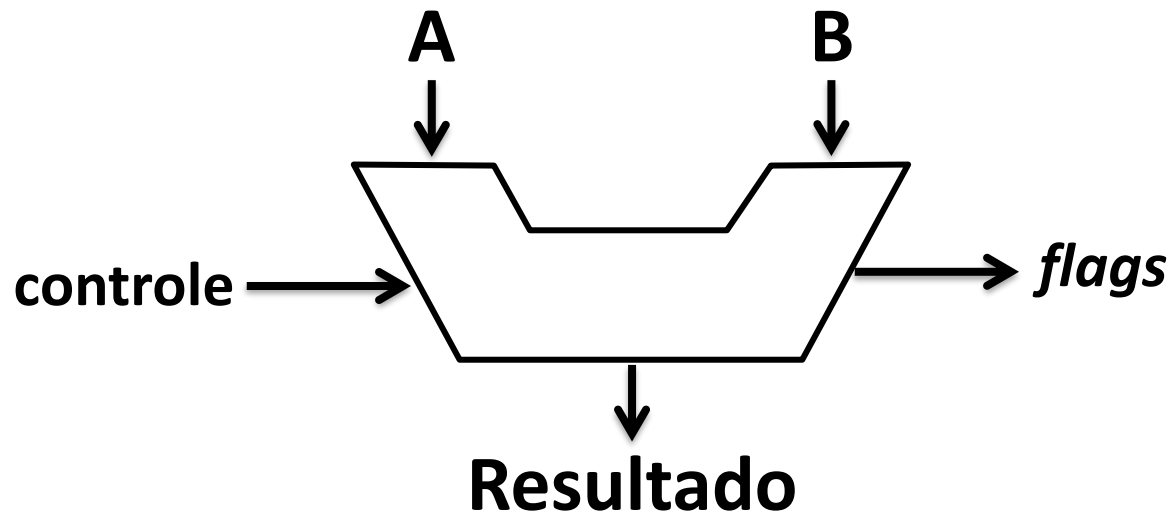
O Harvard Mark 1 (IBM automatic sequence controlled calculator), foi criado em 1943, sendo um sistema eletromecânico à base de relés. Howard Aiken fez o design do hardware do Mark 1, baseado em uma representação e armazenamento numérico decimal.



Howard Aiken

# Unidade Lógica Aritmética (ULA)

ULA ou Arithmetic Logic Unit (**ALU**), é um componente central do caminho de dados da CPU. A **ALU** consiste de um circuito digital capaz de fazer operações aritméticas e lógicas .



# Meio-somador (Half Adder)

Adição binária é uma operação fundamental para quase todas as computações feitas em uma CPU:

| <i>Entradas</i> |          |          | <i>Saídas</i>  |              |
|-----------------|----------|----------|----------------|--------------|
| <b>a</b>        | <b>+</b> | <b>b</b> | <b>soma</b>    | <b>carry</b> |
| 0               | +        | 0        | 0              | 0            |
| 0               | +        | 1        | 1              | 0            |
| 1               | +        | 0        | 1              | 0            |
| 1               | +        | 1        | <sup>1</sup> 0 | 1            |

Operador aritmético de soma.  
Não é o operador lógico OR.



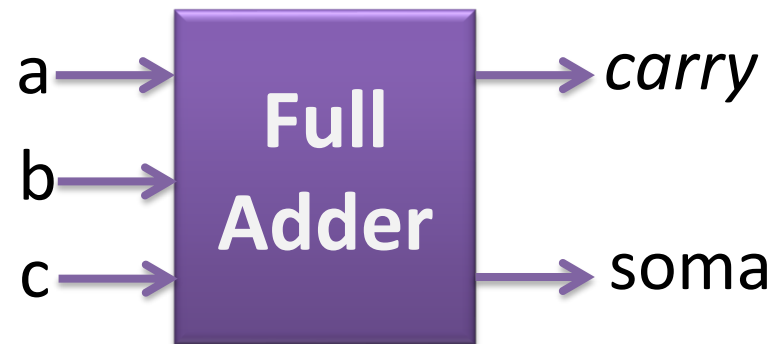
$$S = a \oplus b$$
$$C = a \cdot b$$



# Circuito Aritmético – Adders (Full)

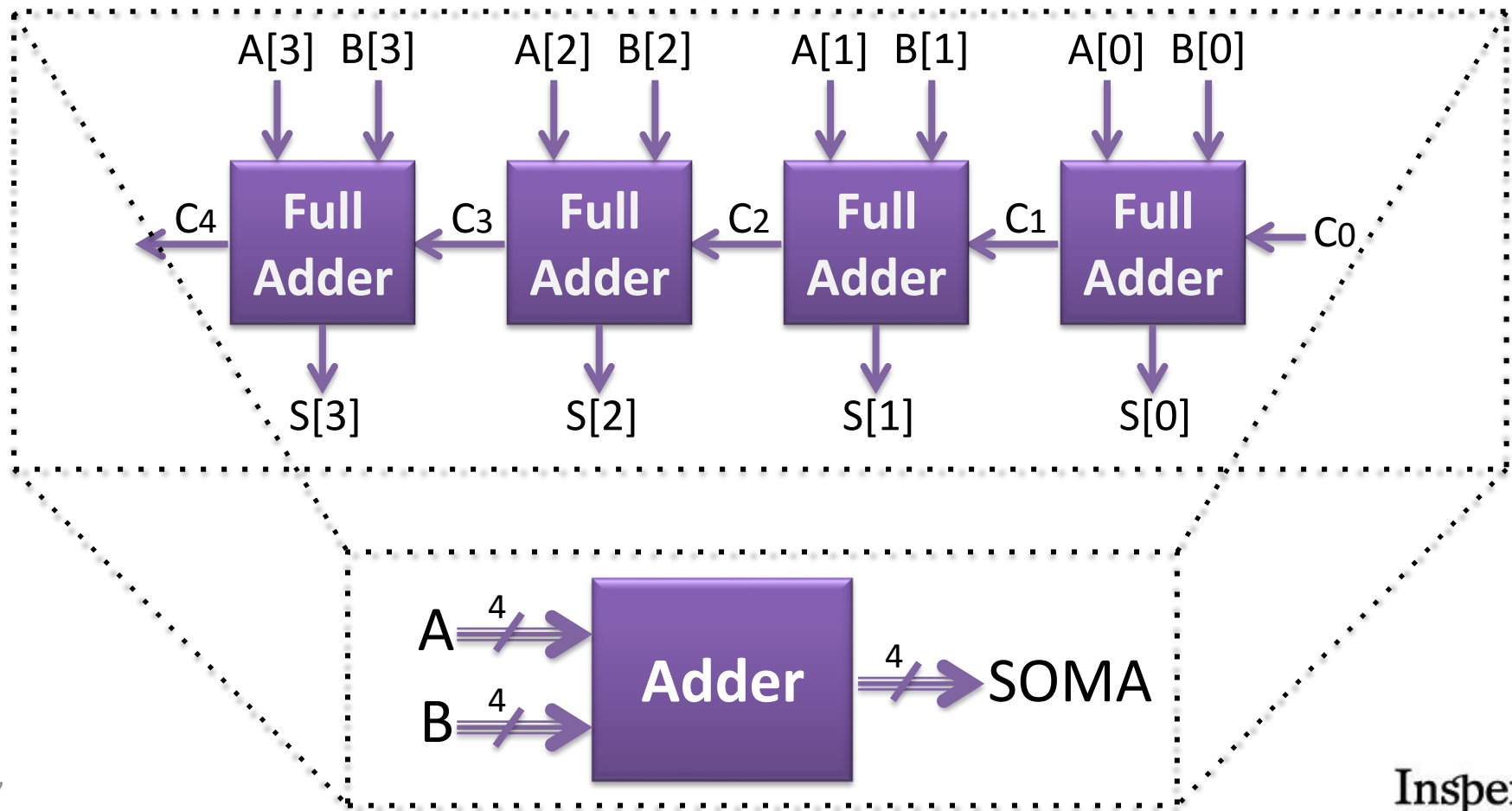
Faz a adição de três bits, retornando a soma e o *carry*.

| <i>Entradas</i> |          |          |          |          | <i>Saídas</i> |              |
|-----------------|----------|----------|----------|----------|---------------|--------------|
| <b>a</b>        | <b>+</b> | <b>b</b> | <b>+</b> | <b>c</b> | <b>soma</b>   | <b>carry</b> |
| 0               | +        | 0        | +        | 0        | 0             | 0            |
| 0               | +        | 0        | +        | 1        | 1             | 0            |
| 0               | +        | 1        | +        | 0        | 1             | 0            |
| 0               | +        | 1        | +        | 1        | 0             | 1            |
| 1               | +        | 0        | +        | 0        | 1             | 0            |
| 1               | +        | 0        | +        | 1        | 0             | 1            |
| 1               | +        | 1        | +        | 0        | 0             | 1            |
| 1               | +        | 1        | +        | 1        | 1             | 1            |



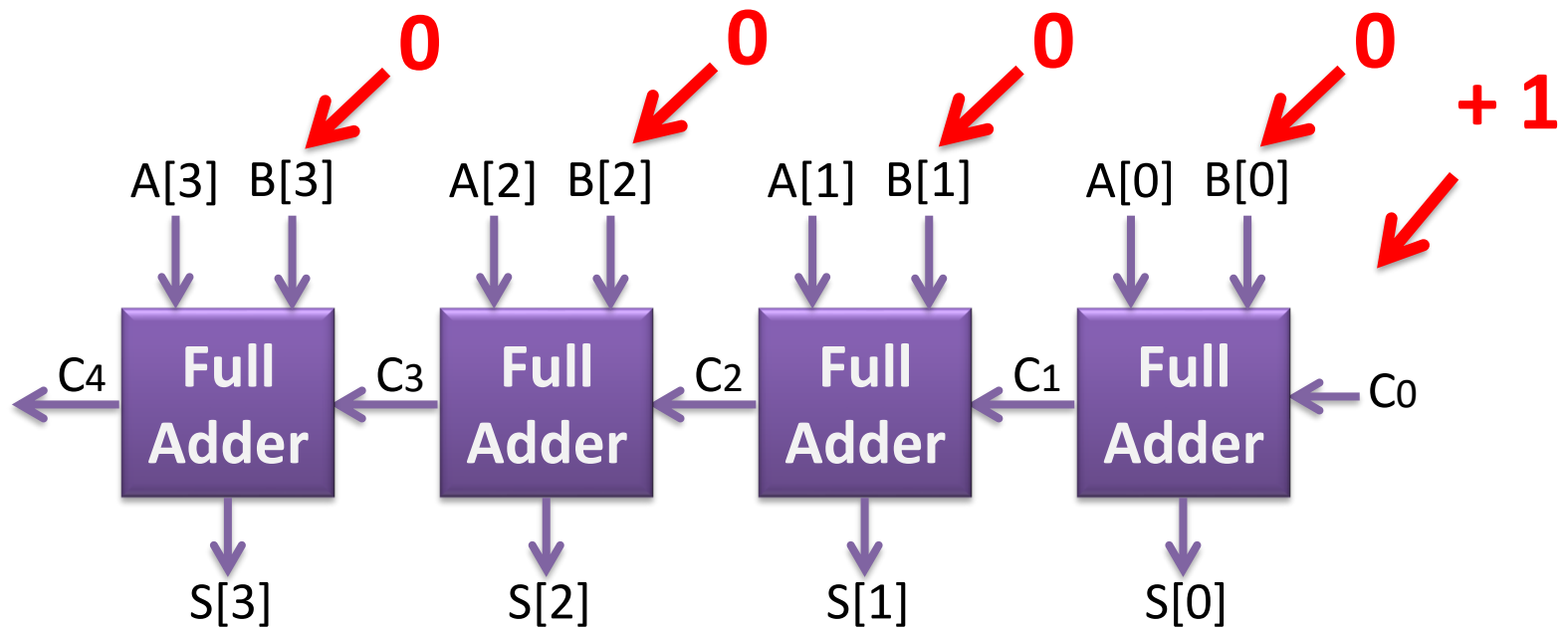
# Circuito Aritmético – Adder

Um circuito para adicionar dois valores de  $n$  bits pode ser construído encadeando  $n$  somadores completos (Full Adders).

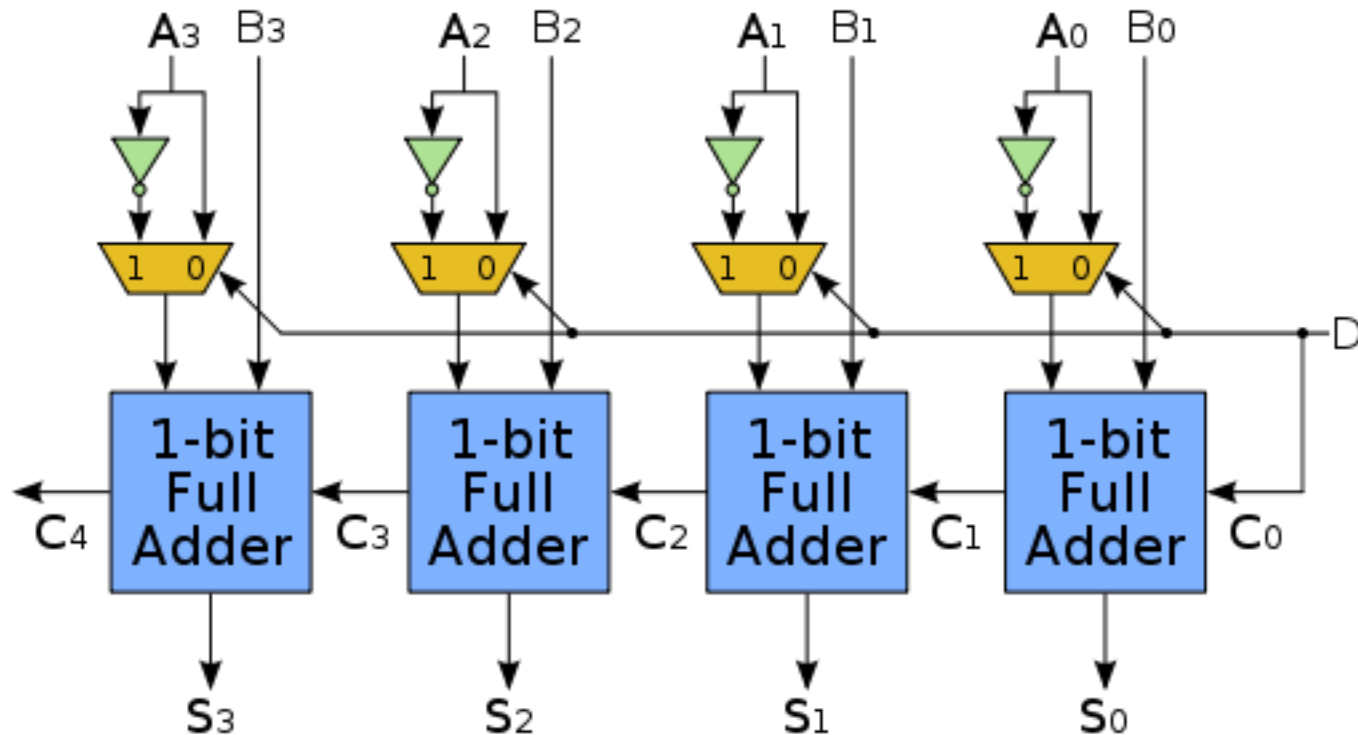


# Circuito Aritmético – Incrementador

Um incrementador, ou seja, um circuito que soma 1 a um valor binário, pode ser usado por exemplo para um contador.



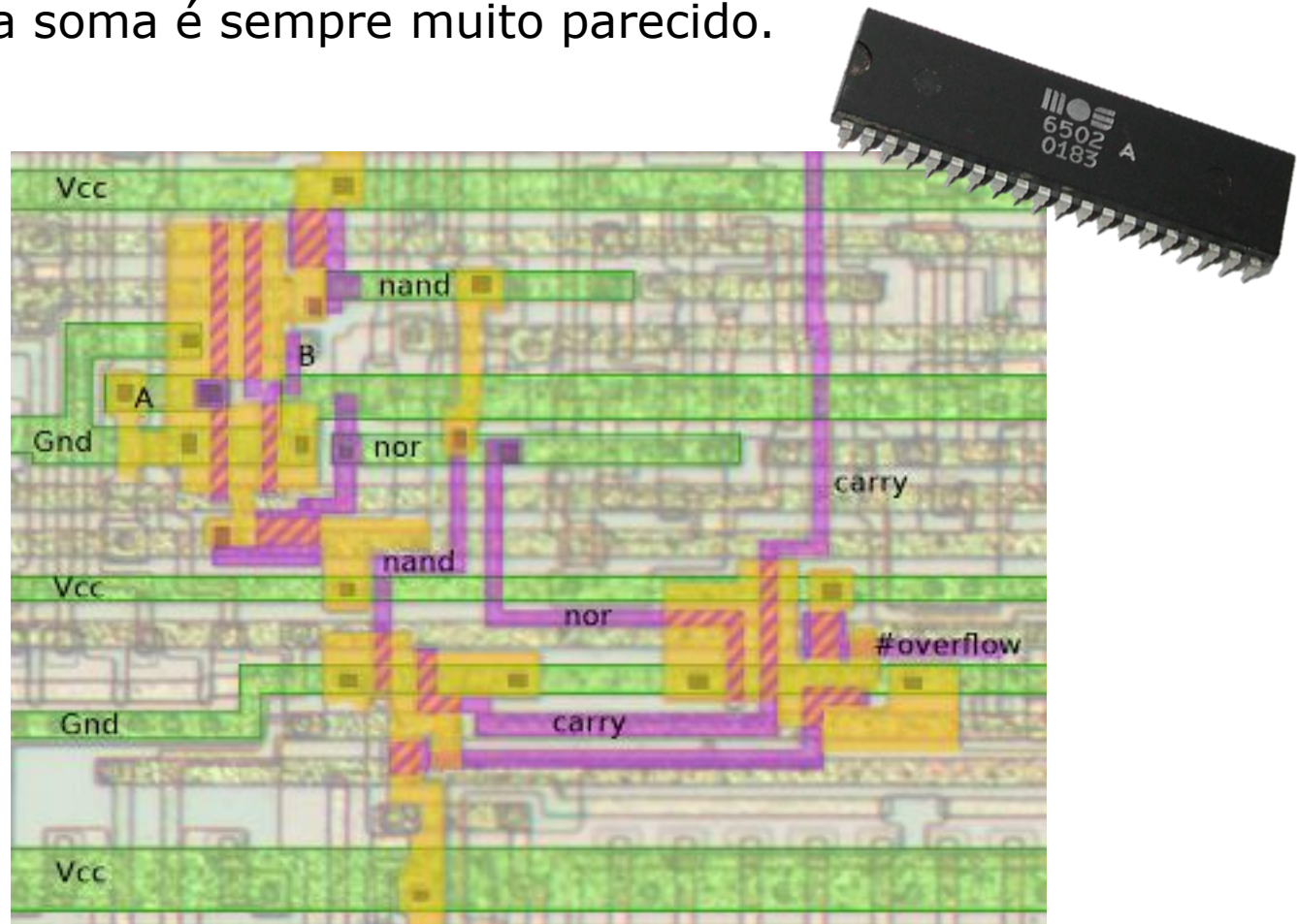
# Somador/Subtrator de 4 bits



somador/subtrator que calcula **B+A** e **B-A**

# Somador em CPU

Diversas otimizações podem ser realizadas, mas o princípio de circuito para soma é sempre muito parecido.



CPU 6502

Ken Shirriff's blog

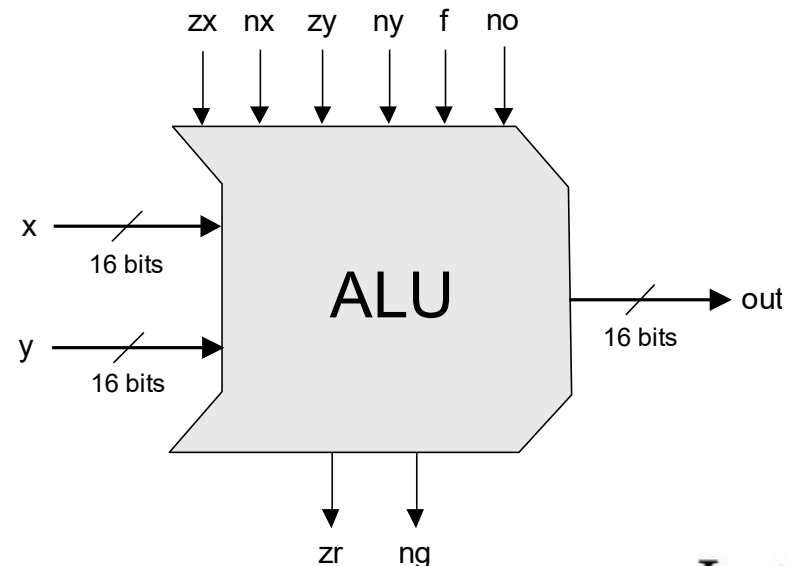
# ALU do Z0 (baseada no Hack)

A arquitetura do Z0 (Hack) é baseada em 16 bits. Ou seja, ele tem de fazer operações com entradas de 16 bits e sair com o valor da operação também em 16 bits. Existem 6 bits de controle e mais dois bits de resultado (flags).

As operações desejadas devem retornar as seguintes saídas:

```
x+y, x-y, y-x,  
0, 1, -1,  
x, y, -x, -y,  
!x, !y,  
x+1, y+1, x-1, y-1,  
x&y, x|y
```

Para isso os seguintes controles podem ser usados:



# Negar e Inverter

A negação é representada com o símbolo – (menos aritmético)

$$\text{Por exemplo } A = 5_{10} = 00000101_2$$

$$-A = -5_{10} = 11111011_2$$

A inversão é representada com o símbolo ! (operador lógico not)

$$\text{Por exemplo } A = 5_{10} = 00000101_2$$

$$!A = 11111010_2 = -6_{10}$$

Também conhecido como **bitwise not**

# Tabela de controle da ALU do Z0 (Hack)

| se zx:<br>x=0 | se nx:<br>x=!x | se zy:<br>y=0 | se ny:<br>y=!y | se f: x+y<br>senão: x&y | se no:<br>out=!out | out |
|---------------|----------------|---------------|----------------|-------------------------|--------------------|-----|
| 1             | 0              | 1             | 0              | 1                       | 0                  | 0   |
| 1             | 1              | 1             | 1              | 1                       | 1                  | 1   |
| 1             | 1              | 1             | 0              | 1                       | 0                  | -1  |
| 0             | 0              | 1             | 1              | 0                       | 0                  | x   |
| 1             | 1              | 0             | 0              | 0                       | 0                  | y   |
| 0             | 0              | 1             | 1              | 0                       | 1                  | !x  |
| 1             | 1              | 0             | 0              | 0                       | 1                  | !y  |
| 0             | 0              | 1             | 1              | 1                       | 1                  | -x  |
| 1             | 1              | 0             | 0              | 1                       | 1                  | -y  |
| 0             | 1              | 1             | 1              | 1                       | 1                  | x+1 |
| 1             | 1              | 0             | 1              | 1                       | 1                  | y+1 |
| 0             | 0              | 1             | 1              | 1                       | 0                  | x-1 |
| 1             | 1              | 0             | 0              | 1                       | 0                  | y-1 |
| 0             | 0              | 0             | 0              | 1                       | 0                  | x+y |
| 0             | 0              | 0             | 1              | 1                       | 1                  | y-x |
| 0             | 0              | 0             | 0              | 0                       | 0                  | x&y |
| 0             | 1              | 0             | 1              | 0                       | 1                  | x y |

# Exemplos de comportamento da ALU

## Exemplo 1

| controle | zx | nx | zy | ny | f ? + : & | no |
|----------|----|----|----|----|-----------|----|
|          | 0  | 0  | 0  | 0  | 1         | 0  |

| x | x[7] | x[6] | x[5] | x[4] | x[3] | x[2] | x[1] | x[0] |      |
|---|------|------|------|------|------|------|------|------|------|
|   | 0    | 0    | 0    | 0    | 1    | 0    | 1    | 0    | } 10 |

| y | y[7] | y[6] | y[5] | y[4] | y[3] | y[2] | y[1] | y[0] |      |
|---|------|------|------|------|------|------|------|------|------|
|   | 1    | 1    | 1    | 1    | 1    | 1    | 1    | 0    | } -2 |

| saída | out[7] | out[6] | out[5] | out[4] | out[3] | out[2] | out[1] | out[0] | zr | ng |
|-------|--------|--------|--------|--------|--------|--------|--------|--------|----|----|
|       | 0      | 0      | 0      | 0      | 1      | 0      | 0      | 0      | 0  | 0  |

8

# Exemplos de comportamento da ALU

## Exemplo 2

| controle | zx | nx | zy | ny | f ? + : & | no |
|----------|----|----|----|----|-----------|----|
|          | 0  | 0  | 1  | 1  | 1         | 0  |

| x | x[7] | x[6] | x[5] | x[4] | x[3] | x[2] | x[1] | x[0] |      |
|---|------|------|------|------|------|------|------|------|------|
|   | 0    | 0    | 0    | 0    | 1    | 0    | 1    | 0    | } 10 |

| y | y[7] | y[6] | y[5] | y[4] | y[3] | y[2] | y[1] | y[0] |      |
|---|------|------|------|------|------|------|------|------|------|
|   | 1    | 1    | 1    | 1    | 1    | 1    | 1    | 0    | } -2 |

| saída | out[7] | out[6] | out[5] | out[4] | out[3] | out[2] | out[1] | out[0] | zr | ng |
|-------|--------|--------|--------|--------|--------|--------|--------|--------|----|----|
|       | 0      | 0      | 0      | 0      | 1      | 0      | 0      | 1      | 0  | 0  |

9

# Exemplos de comportamento da ALU

## Exemplo 3

| controle | zx | nx | zy | ny | f ? + : & | no |
|----------|----|----|----|----|-----------|----|
|          | 0  | 0  | 1  | 1  | 1         | 1  |

| x | x[7] | x[6] | x[5] | x[4] | x[3] | x[2] | x[1] | x[0] |      |
|---|------|------|------|------|------|------|------|------|------|
|   | 0    | 0    | 0    | 0    | 1    | 0    | 1    | 0    | } 10 |

| y | y[7] | y[6] | y[5] | y[4] | y[3] | y[2] | y[1] | y[0] |      |
|---|------|------|------|------|------|------|------|------|------|
|   | 1    | 1    | 1    | 1    | 1    | 1    | 1    | 0    | } -2 |

| saída | out[7] | out[6] | out[5] | out[4] | out[3] | out[2] | out[1] | out[0] | zr | ng |
|-------|--------|--------|--------|--------|--------|--------|--------|--------|----|----|
|       | 1      | 1      | 1      | 1      | 0      | 1      | 1      | 0      | 0  | 1  |

-10

# Exemplos de comportamento da ALU

## Exemplo 4

| controle | zx | nx | zy | ny | f ? + : & | no |
|----------|----|----|----|----|-----------|----|
|          | 1  | 0  | 1  | 0  | 1         | 0  |

| x | x[7] | x[6] | x[5] | x[4] | x[3] | x[2] | x[1] | x[0] |      |
|---|------|------|------|------|------|------|------|------|------|
|   | 0    | 0    | 0    | 0    | 1    | 0    | 1    | 0    | } 10 |

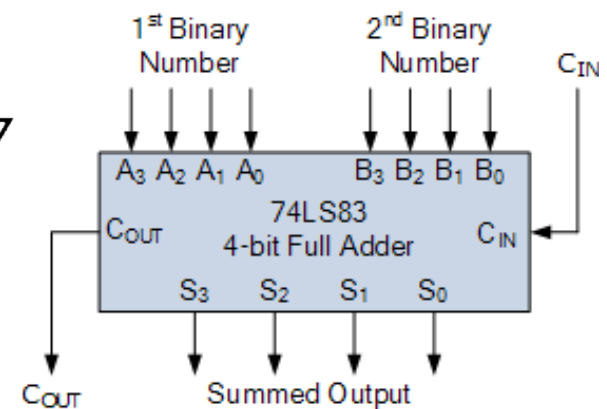
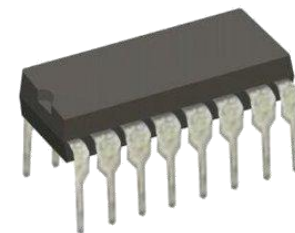
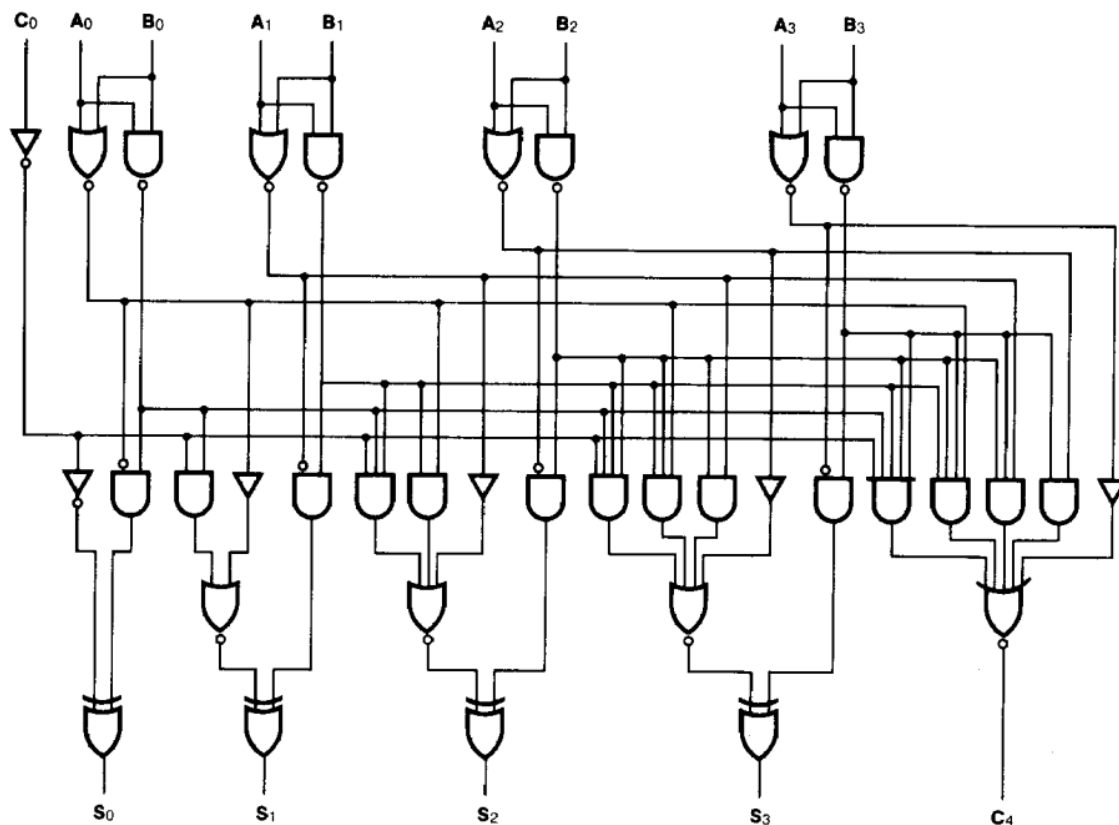
| y | y[7] | y[6] | y[5] | y[4] | y[3] | y[2] | y[1] | y[0] |      |
|---|------|------|------|------|------|------|------|------|------|
|   | 1    | 1    | 1    | 1    | 1    | 1    | 1    | 0    | } -2 |

| saída | out[7] | out[6] | out[5] | out[4] | out[3] | out[2] | out[1] | out[0] | zr | ng |
|-------|--------|--------|--------|--------|--------|--------|--------|--------|----|----|
|       | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 1  | 0  |

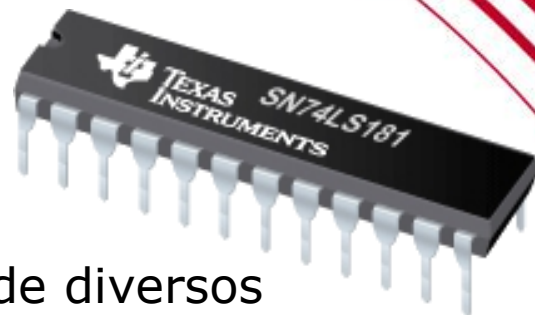
0

# 7483

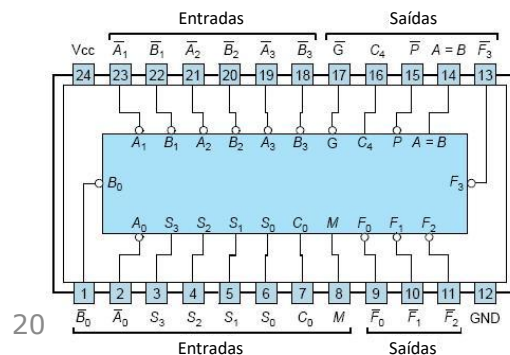
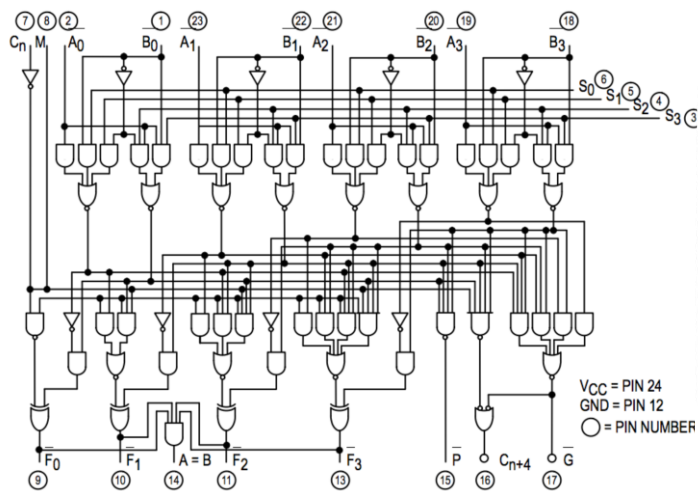
O 7483 é um circuito integrado somador de 4-bits com *carry lookahead*, tendo *carry* de entrada e saída.



# 74181



O 74181 foi utilizado na implementação da ALU de diversos computadores, como o PDP-11 e o Nova, sendo a primeira unidade lógica aritmética em um único chip, equivalente a 75 portas lógicas.



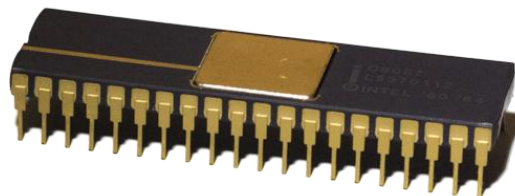
| SELECTION |    |    |    | ACTIVE-LOW DATA             |                                                      |                                                      |
|-----------|----|----|----|-----------------------------|------------------------------------------------------|------------------------------------------------------|
|           |    |    |    | M = H<br>LOGIC<br>FUNCTIONS | M = L; ARITHMETIC OPERATIONS<br>Cn = L<br>(no carry) | Cn = H<br>(with carry)                               |
| S3        | S2 | S1 | S0 |                             |                                                      |                                                      |
| L         | L  | L  | L  | $F = \bar{A}$               | $F = A \text{ MINUS } 1$                             | $F = A$                                              |
| L         | L  | L  | H  | $F = \bar{A}\bar{B}$        | $F = AB \text{ MINUS } 1$                            | $F = AB$                                             |
| L         | L  | H  | L  | $F = \bar{A} + B$           | $F = A\bar{B} \text{ MINUS } 1$                      | $F = A\bar{B}$                                       |
| L         | L  | H  | H  | $F = 1$                     | $F = \text{MINUS } 1 \text{ (2's COMP)}$             | $F = \text{ZERO}$                                    |
| L         | H  | L  | L  | $F = \bar{A} + \bar{B}$     | $F = A \text{ PLUS } (A + \bar{B})$                  | $F = A \text{ PLUS } (A + \bar{B}) \text{ PLUS } 1$  |
| L         | H  | L  | H  | $F = \bar{B}$               | $F = AB \text{ PLUS } (A + \bar{B})$                 | $F = AB \text{ PLUS } (A + \bar{B}) \text{ PLUS } 1$ |
| L         | H  | H  | L  | $F = A \oplus B$            | $F = A \text{ MINUS } B \text{ MINUS } 1$            | $F = A \text{ MINUS } B$                             |
| L         | H  | H  | H  | $F = A + \bar{B}$           | $F = A + \bar{B}$                                    | $F = (A + \bar{B}) \text{ PLUS } 1$                  |
| H         | L  | L  | L  | $F = \bar{A}B$              | $F = A \text{ PLUS } (A + B)$                        | $F = A \text{ PLUS } (A + B) \text{ PLUS } 1$        |
| H         | L  | L  | H  | $F = A \oplus B$            | $F = A \text{ PLUS } B$                              | $F = A \text{ PLUS } B \text{ PLUS } 1$              |
| H         | L  | H  | L  | $F = B$                     | $F = A\bar{B} \text{ PLUS } (A + B)$                 | $F = A\bar{B} \text{ PLUS } (A + B) \text{ PLUS } 1$ |
| H         | L  | H  | H  | $F = A + B$                 | $F = (A + B)$                                        | $F = (A + B) \text{ PLUS } 1$                        |
| H         | H  | L  | L  | $F = 0$                     | $F = A \text{ PLUS } A^\dagger$                      | $F = A \text{ PLUS } A \text{ PLUS } 1$              |
| H         | H  | L  | H  | $F = \bar{A}\bar{B}$        | $F = AB \text{ PLUS } A$                             | $F = AB \text{ PLUS } A \text{ PLUS } 1$             |
| H         | H  | H  | L  | $F = AB$                    | $F = A\bar{B} \text{ PLUS } A$                       | $F = A\bar{B} \text{ PLUS } A \text{ PLUS } 1$       |
| H         | H  | H  | H  | $F = A$                     | $F = A$                                              | $F = A \text{ PLUS } 1$                              |

# Unidade de Ponto Flutuante

Uma unidade de ponto flutuante, ou FPU (*Floating-Point Unit*), é um circuito especializado na execução de operações com números representados em ponto flutuante. Ela pode existir como um coprocessador separado ou estar integrada ao processador principal.



AMD AM9511



Intel 8087



Motorola 68881

# Insper

[www.insper.edu.br](http://www.insper.edu.br)